

JigChat V3

Full-Stack Real-Time Chat Application — Technical Documentation

Version 3.0 May 2026 Node.js · React 19 · PostgreSQL · Redis · Socket.IO

TABLE OF CONTENTS

1. Project Overview
2. Tech Stack
3. High-Level Architecture
4. Database Schema
5. Backend Architecture
 1. Directory Structure
 2. Middleware Chain
 3. REST API Reference
 4. Socket.IO Events
 5. Security & Encryption
6. Frontend Architecture
 1. Directory Structure
 2. State Management
 3. Component Tree
 4. Socket Hook
 5. API Client
7. Workflow Diagrams
8. Sequence Diagrams
9. Environment Variables
10. Key Features

1. Project Overview

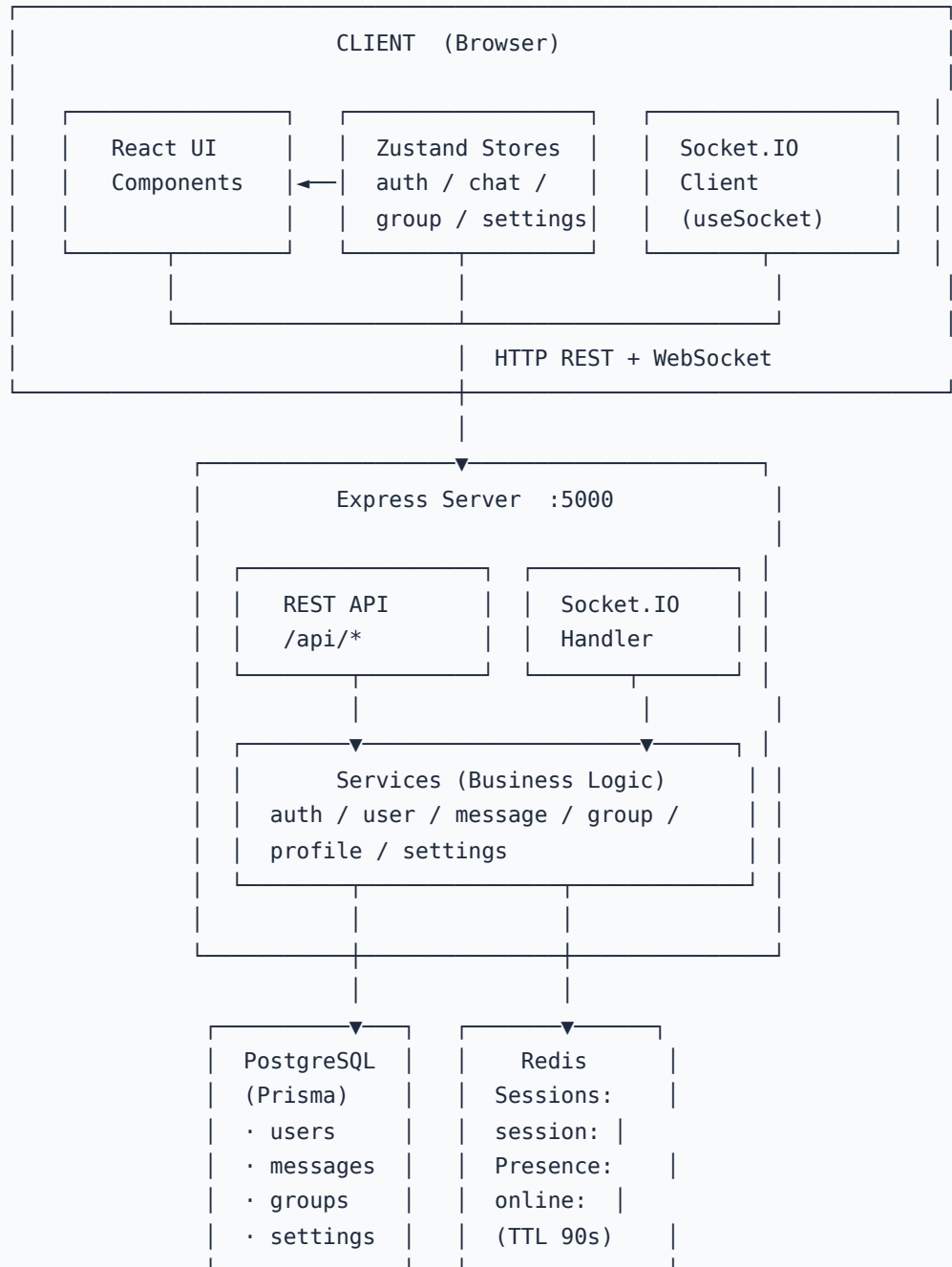
JigChat V3 is a production-ready, full-stack real-time messaging web application. It supports one-on-one direct messaging, group chats with admin controls, live online presence, typing indicators, and message encryption at rest using AES-256-GCM.

Capability	Detail
Messaging	1-on-1 DMs and multi-user group chats
Real-time	Socket.IO — messages, typing, presence
Encryption	All message content encrypted at rest (AES-256-GCM)
Auth	JWT + Redis-backed sessions
Presence	Redis TTL keys with 30-second heartbeat
UI	Responsive (desktop + mobile), Tailwind CSS + Radix UI

2. Tech Stack

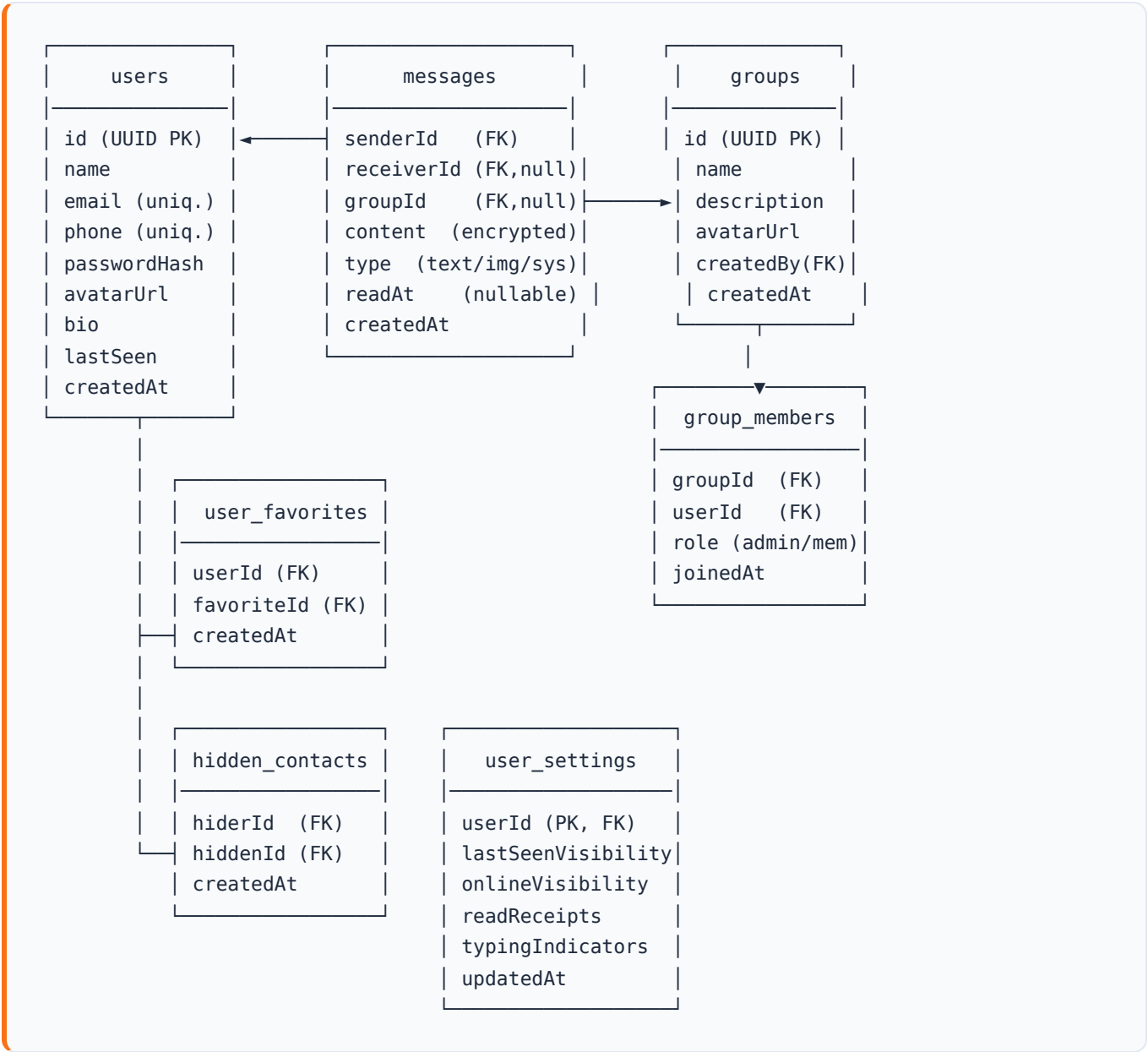
Layer	Technology	Version
Runtime	Node.js	v20 (ESNext)
Server Framework	Express	5.x
Real-time	Socket.IO	4.x
Database	PostgreSQL + Prisma ORM	Prisma 7.x
Cache / Sessions	Redis (ioredis)	5.x
Auth	jsonwebtoken + bcryptjs	JWT 9.x
Encryption	Node.js crypto (AES-256-GCM)	built-in
Frontend	React + TypeScript	React 19
Build Tool	Vite	6.x
Styling	Tailwind CSS + Radix UI (shadcn)	Tailwind 4
State	Zustand	5.x
HTTP Client	Axios	1.x
Routing	React Router	v7

3. High-Level Architecture



4. Database Schema

Entity Relationship Diagram



Model Summary

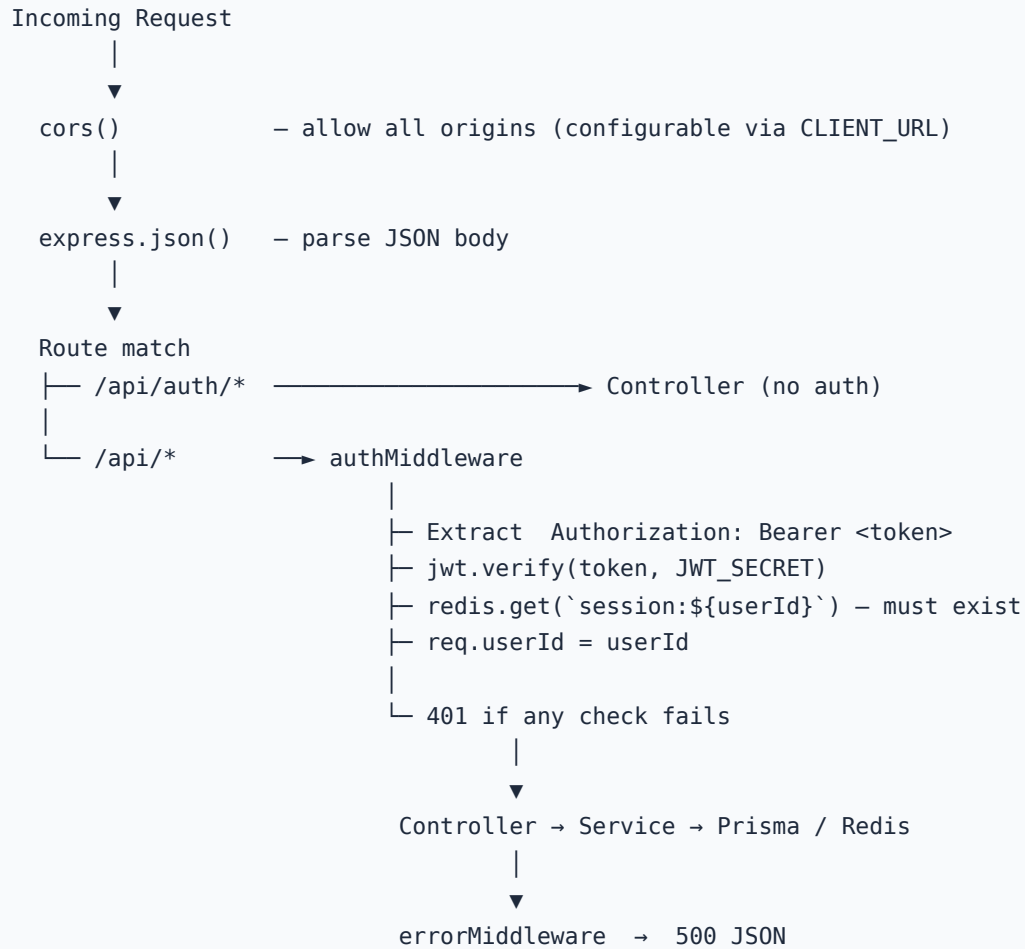
Model	Primary Key	Key Fields
User	UUID	name, email?, phone?, passwordHash, avatarUrl, bio, lastSeen
Message	UUID	senderId, receiverId?, groupId?, content (AES encrypted), type, readAt
Group	UUID	name, description, avatarUrl, createdBy
GroupMember	(groupId, userId)	role: admin member, joinedAt
Favorite	(userId, favoriteId)	createdAt
HiddenContact	(hiderId, hiddenId)	createdAt
UserSettings	userId	lastSeenVisibility, onlineVisibility, readReceipts, typingIndicators

5. Backend Architecture

5.1 Directory Structure

```
server/
├── src/
│   ├── server.ts                # Express + Socket.IO bootstrap (port 5000)
│   ├── controllers/
│   │   ├── auth.controller.ts   # login, logout
│   │   ├── user.controller.ts   # list users, get by id, favorite, hide
│   │   ├── message.controller.ts # DM history, group history, search, delete
│   │   ├── group.controller.ts  # CRUD groups, add/remove members
│   │   ├── profile.controller.ts # view/edit profile, change password
│   │   └── settings.controller.ts # get/patch user settings
│   ├── services/                # Business logic – called by controllers
│   │   ├── auth.service.ts
│   │   ├── user.service.ts
│   │   ├── message.service.ts
│   │   ├── group.service.ts
│   │   ├── profile.service.ts
│   │   └── settings.service.ts
│   ├── routes/                  # Express Router definitions
│   │   ├── auth.routes.ts
│   │   ├── user.routes.ts
│   │   ├── message.routes.ts
│   │   ├── group.routes.ts
│   │   ├── profile.routes.ts
│   │   └── settings.routes.ts
│   ├── middleware/
│   │   ├── auth.middleware.ts    # JWT verify + Redis session check
│   │   └── error.middleware.ts   # Global error handler → 500
│   ├── socket/
│   │   ├── socket.handler.ts     # send_message, send_group_message, typing
│   │   ├── socket.rooms.ts       # join_room, disconnect
│   │   └── socket.auth.ts        # handshake JWT verification
│   └── lib/
│       ├── prisma.ts             # PrismaClient singleton (pg adapter)
│       ├── redis.ts              # ioredis singleton
│       ├── io.ts                 # Socket.IO singleton getter/setter
│       └── crypto.ts             # AES-256-GCM encrypt / decrypt
└── prisma/
    └── schema.prisma
```


5.2 Middleware Chain



5.3 REST API Reference

AUTHENTICATION

Method	Path	Auth	Body	Response
POST	/api/auth/login	No	{ email, password }	{ user, token }
POST	/api/auth/logout	Yes	—	{ message }

USERS

Method	Path	Query / Body	Response
GET	/api/users	?currentUserId & filter	User[] with status, unread, isFavorite
GET	/api/users/:id	—	Public profile (name, avatar, bio, isOnline)
POST	/api/users/favorite	{ userId, favoriteId }	{ isFavorite: boolean }
POST	/api/users/hide	{ hiddenId }	{ message }

MESSAGES

Method	Path	Description
GET	/api/messages/:senderId/:receiverId	Full DM history (decrypted, ordered by createdAt asc)
GET	/api/messages/group/:groupId	Group history with sender name + avatar
POST	/api/messages/mark-read	Sets readAt on all unread messages from sender
DELETE	/api/messages/:messageId	Sender-only delete; broadcasts message_deleted via Socket
GET	/api/messages/search?q=¤tUserId=	ILIKE search — top 5 contacts + top 8 message matches

GROUPS

Method	Path	Description
POST	<code>/api/groups</code>	Create group with name, description, memberIds (creator = admin)
GET	<code>/api/groups?userId=</code>	All groups user is a member of, sorted by createdAt desc
GET	<code>/api/groups/:groupId</code>	Single group with full member details
PUT	<code>/api/groups/:groupId</code>	Admin only — update name / description
POST	<code>/api/groups/:groupId/members</code>	Admin only — add member; broadcasts member_added socket event
DELETE	<code>/api/groups/:groupId/members/:userId</code>	Admin or self — remove member; broadcasts removal events

PROFILE & SETTINGS

Method	Path	Description
GET	<code>/api/profile</code>	Current user full profile
PUT	<code>/api/profile</code>	Update name, bio, avatarUrl
PUT	<code>/api/profile/password</code>	Change password (bcrypt 12 rounds)
GET	<code>/api/settings/:userId</code>	Get or create default UserSettings
PATCH	<code>/api/settings/:userId</code>	Update any settings field (visibility, receipts, typing)

5.4 Socket.IO Events

CLIENT → SERVER

Event	Payload	Server Action
join_room	—	SET online:<id> EX 90, join group rooms, broadcast user_online
join_group_room	groupId	socket.join("group:<id>")
send_message	{ senderId, receiverId, content, type }	Encrypt, INSERT, broadcast receive_message to both
send_group_message	{ senderId, groupId, content, type }	Verify member, encrypt, INSERT, broadcast to room
typing	{ senderId, receiverId }	Forward user_typing to receiver
stop_typing	{ senderId, receiverId }	Forward user_stop_typing to receiver

SERVER → CLIENT

Event	Payload	Triggered By
receive_message	Message (decrypted)	New DM — sent to both participants
receive_group_message	Message + sender info	New group message — broadcast to room
message_deleted	{ messageId }	Message deletion by sender
user_online	userId	User connects / heartbeat
user_offline	{ userId, lastSeen }	User disconnects
user_typing	userId	Typing event forwarded
user_stop_typing	userId	Stop typing forwarded
member_added	{ group, systemMessage }	Sent to newly added user
member_removed	{ groupId, userId }	Sent to removed user
group_member_removed	{ groupId, userId }	Sent to remaining members
group_send_error	{ groupId, message }	Non-member tried to send

REDIS KEY PATTERNS

Key	Value	TTL	Purpose
<code>session:<userId></code>	JWT token	7 days	Active session — deleted on logout
<code>online:<userId></code>	"1"	90 seconds	Online presence — refreshed every 30s

5.5 Security & Encryption

Password Storage

bcryptjs · 12 rounds · one-way hash stored as password_hash

JWT Tokens

Algorithm : HS256

Expiry : 7 days

Validated : `jwt.verify()` + `redis.get('session:<id>')` must exist

Logout : `DEL session:<id>` in Redis → token immediately invalid

Message Encryption (AES-256-GCM)

Key : 32 bytes from ENCRYPTION_KEY env var

IV : 12 bytes (random per message)

Auth tag : 16 bytes

Format : "ENC:<iv_b64>:<tag_b64>:<ciphertext_b64>"

Legacy : Graceful fallback – plaintext returned if not ENC: prefixed

Socket Authentication

JWT required in `socket.handshake.auth.token` before any events accepted

Group Authorization

`GroupMember.role = 'admin' | 'member'`

Admin-only: update group info, add member, remove other members

Self: any member can remove themselves

6. Frontend Architecture

6.1 Directory Structure

```
frontend/src/
├─ main.tsx                # ReactDOM root
├─ App.tsx                 # Routes: /login /chat / (redirect)
├─ pages/
│   ├─ LoginPage.tsx       # Email + password form → POST /auth/login
│   └─ ChatPage.tsx        # Main layout orchestrator
├─ components/
│   ├─ chat/
│   │   ├─ TopHeader.tsx   # Search + settings / profile buttons
│   │   ├─ SidebarHeader.tsx # Tab filter (All/Unread/Favorites/Groups)
│   │   ├─ ContactList.tsx  # User + group list, unread badges
│   │   ├─ ChatHeader.tsx   # Active chat: avatar, status, actions
│   │   ├─ MessageList.tsx  # Scrollable thread, date separators
│   │   ├─ MessageInput.tsx # Textarea + emoji + typing emit
│   │   ├─ SearchBar.tsx    # Global search (contacts + messages)
│   │   ├─ UserProfilePanel.tsx # Right-side slide-in profile panel
│   │   └─ WelcomeScreen.tsx # Empty state when no chat selected
│   ├─ groups/
│   │   └─ GroupModal.tsx    # Create / edit group + member management
│   └─ profile/
│       └─ ProfileModal.tsx  # Edit profile, change password
├─ settings/
│   └─ SettingsModal.tsx    # Privacy toggles (visibility, receipts)
├─ shared/
│   ├─ Avatar.tsx           # Image with initials fallback
│   ├─ AvatarViewer.tsx     # Fullscreen viewer: zoom + pan + reset
│   ├─ Button.tsx           # default / ghost / outline variants
│   ├─ Dialog.tsx           # Modal wrapper
│   ├─ Input.tsx            # Styled text input
│   ├─ Textarea.tsx         # Multi-line input
│   ├─ Switch.tsx           # Toggle
│   └─ Dropdown.tsx         # Menu
├─ store/
│   ├─ authStore.ts         # user · isAuthenticated · token (persisted)
│   ├─ chatStore.ts         # messages · users · selectedUser · UI state
│   ├─ groupStore.ts        # groups · modal state · favorites
│   └─ settingsStore.ts     # UserSettings
├─ hooks/
│   └─ useSocket.ts         # Singleton socket + all event handlers
├─ api/
│   ├─ client.ts            # Axios instance + auth interceptor
│   └─ settings.ts          # Settings fetch/update helpers
```

6.2 State Management (Zustand)

```
authStore                                     [localStorage persist]
┌───────────────────────────────────────────┐
│ user · isAuthenticated · token           │
│ login() · logout() · updateUser() · clearAuth() │
```

```
chatStore
┌───────────────────────────────────────────┐
│ users[]           – contact list with status │
│ messages[]        – active DM thread         │
│ groupMessages[]   – active group thread       │
│ selectedUser      – current DM partner       │
│ selectedGroup     – current group            │
│ groupUnreadCounts – { groupId: number }      │
│ groupLastMessages – { groupId: { content, createdAt } } │
│ profilePanelUser  – user shown in right panel │
│ highlightedMsgId  – flash-highlight from search jump │
│ activeTab         – All / Unread / Favorites / Groups │
│ isSettingsOpen · isProfileOpen
```

```
groupStore
┌───────────────────────────────────────────┐
│ groups[]          – user's group list        │
│ isModalOpen       – GroupModal visibility    │
│ editingGroup      – group being edited in modal │
│ favoriteGroupIds[] – client-side starred groups
```

6.3 Component Tree

```
App
├── ChatPage
│   ├── TopHeader
│   │   └── SearchBar — contacts + message search results
│   ├── Sidebar
│   │   ├── SidebarHeader — tab filter
│   │   └── ContactList
│   │       ├── "Saved Messages" (self-chat)
│   │       ├── User rows (avatar · name · last msg · unread badge)
│   │       └── Group rows (icon · name · last msg · unread badge)
│   ├── Chat Area
│   │   ├── ChatHeader (avatar | name | online status | action buttons)
│   │   │   └── [click avatar/name] → setProfilePanelUser()
│   │   ├── Content Row
│   │   │   ├── MessageList
│   │   │   │   ├── DateSeparator
│   │   │   │   ├── SystemMessage pill
│   │   │   │   └── Message bubble (sent / received)
│   │   │   │       └── sender avatar + name (group, clickable)
│   │   │   │           └── [click] → setProfilePanelUser()
│   │   │   ├── MessageInput (textarea · emoji · send · typing emit)
│   │   │   └── UserProfilePanel [conditional right column]
│   │   │       ├── avatar [click] → AvatarViewer (fullscreen)
│   │   │       │   └── ZoomIn · ZoomOut · Reset · Drag-to-pan
│   │   │       ├── name · online status
│   │   │       └── bio · email · phone (if available)
│   │   └── WelcomeScreen (no active chat)
│   ├── SettingsModal
│   ├── ProfileModal
│   └── GroupModal
```


6.4 Socket Hook (useSocket)

```
useSocket() ← called once in ChatPage
```

- Creates singleton Socket.IO client (persists across re-renders)
 - URL: VITE_SOCKET_URL
 - Auth: { token } in handshake
 - Reconnection: 5 attempts · 1s delay · websocket + polling
- On connect: emit join_room
- Heartbeat: emit join_room every 30 seconds
(refreshes Redis online:<id> TTL before it expires at 90s)
- Inbound event handlers:
 - receive_message → addMessage() · markAsRead() · bumpUserToTop()
 - receive_group_message → addGroupMessage() · bumpGroupLastMessage()
 - message_deleted → removeMessage()
 - member_added → addGroupLocally() · join socket room
 - member_removed → removeGroupLocally()
 - group_member_removed → removeMemberLocally() · update selectedGroup
 - user_online → updateUserStatus(id, true)
 - user_offline → updateUserStatus(id, false, lastSeen)
 - user_typing → updateUserTypingStatus(id, true)
 - user_stop_typing → updateUserTypingStatus(id, false)
- Outbound helpers:
 - sendMessage(receiverId, content) → emit send_message
 - sendGroupMessage(groupId, content) → emit send_group_message
 - emitTyping(receiverId) → emit typing (auto-stop 2s)
 - emitStopTyping(receiverId) → emit stop_typing

6.5 API Client

```
Axios instance (baseUrl = VITE_API_BASE_URL)
```

- Request interceptor
 - authStore.getState().token
 - set Authorization: Bearer <token>
- Response interceptor
 - on 401 → authStore.clearAuth() → redirect to "/"

7. Workflow Diagrams

7.1 Authentication Flow



7.2 Direct Message Flow

User A selects User B



GET /api/messages/:A/:B → messages[] in chatStore



User A types

- └─ emit "typing" → server → emit "user_typing" to B
- └─ auto emit "stop_typing" after 2 seconds of no keys



User A presses Enter



emit "send_message" { senderId:A, receiverId:B, content }



Server:

1. AES-256-GCM encrypt content
2. INSERT into messages
3. Un-hide contact if previously hidden
4. emit "receive_message" → User A + User B



User A

User B

addMessage() └─ Active chat with A?

- └─ YES → addMessage() + POST mark-read
- └─ NO → incrementUnreadCount(A)
bumpUserToTop(A)

7.3 Group Message Flow

```
Admin creates group
  POST /api/groups { name, memberIds }
    | creates GroupMember rows (creator = admin)
    |
    emit "join_group_room" groupId
    | all members socket.join("group:<id>")
    |
    ▼
Member sends message
  emit "send_group_message" { senderId, groupId, content }
    |
    ▼
Server:
  1. Verify sender is GroupMember
  2. AES-256-GCM encrypt
  3. INSERT into messages
  4. emit "receive_group_message" to room "group:<id>"
    |
    ▼
All room members receive:
  └─ Active in group → addGroupMessage()
  └─ Elsewhere      → incrementGroupUnreadCount()
                     bumpGroupLastMessage()
```

7.4 Online Presence Flow

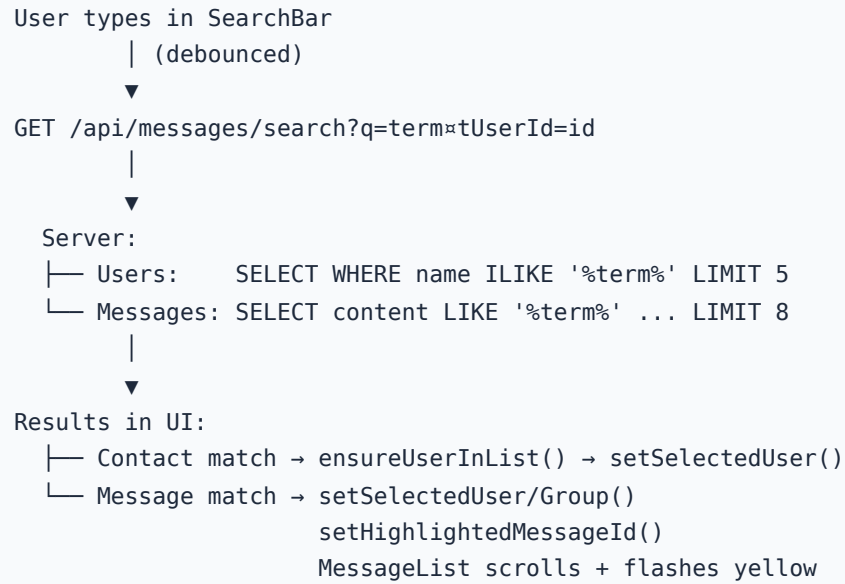
```
Socket connects
  | emit "join_room"
  |
  ▼
Server:
  redis.set(`online:${id}`, "1", "EX", 90)
  socket.join all group rooms
  emit "user_online" id → all sockets

  ── every 30 seconds ───────────────────
Client re-emits "join_room"
Server: redis.set(`online:${id}`, "1", "EX", 90) ← refresh TTL

  ── on disconnect ───────────────────
Server:
  redis.del(`online:${id}`)
  UPDATE users SET lastSeen = now() WHERE id = ?
  emit "user_offline" { id, lastSeen } → all sockets

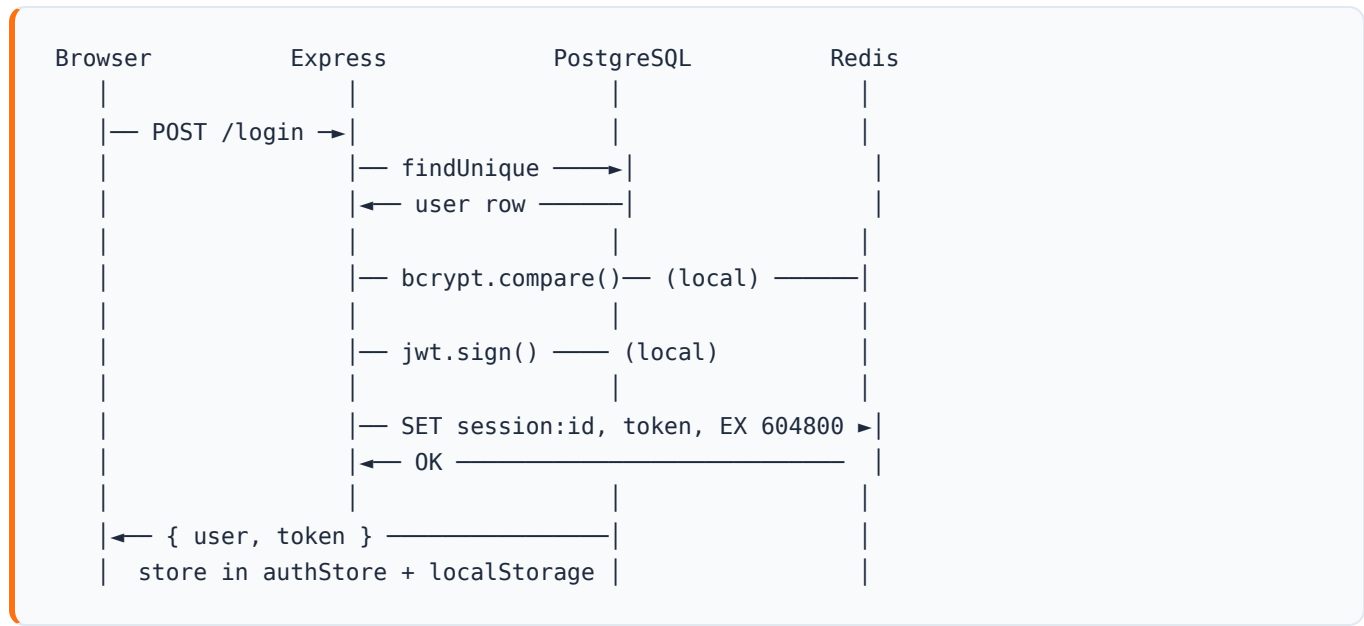
All clients: updateUserStatus(id, false, lastSeen)
```

7.5 Search Flow

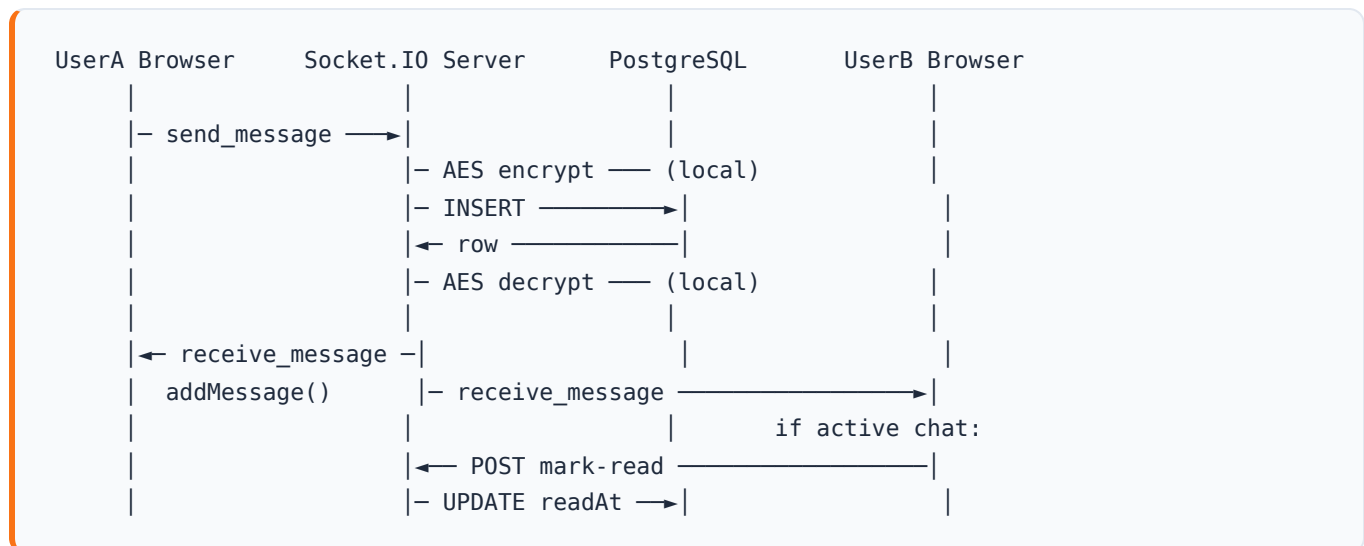


8. Sequence Diagrams

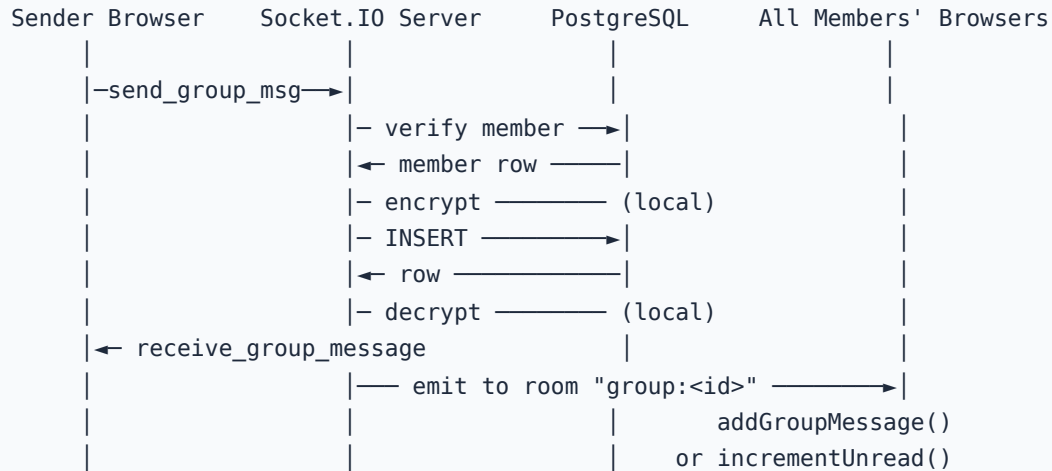
8.1 Login Sequence



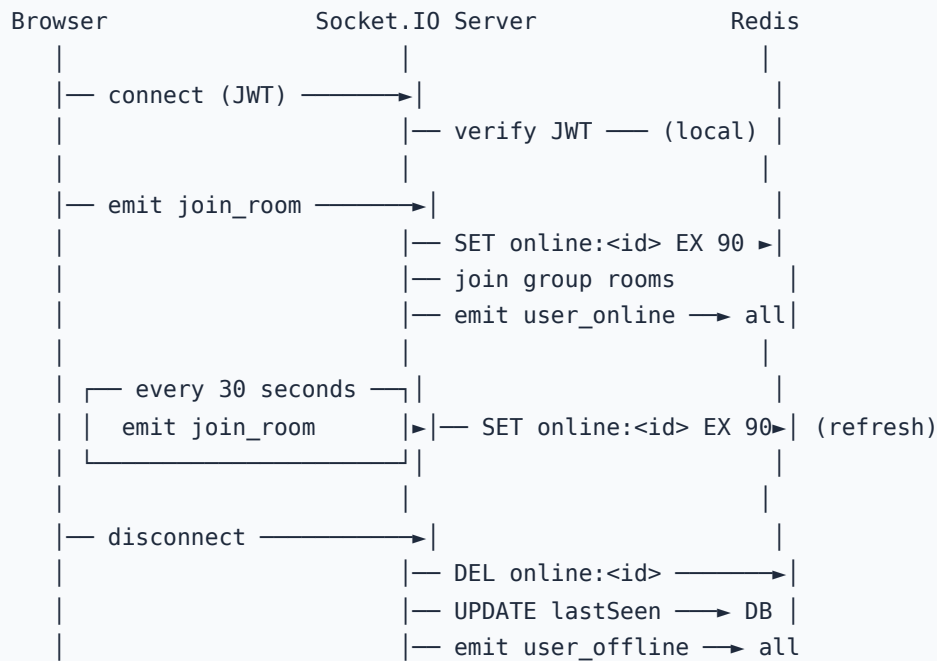
8.2 Send DM Sequence



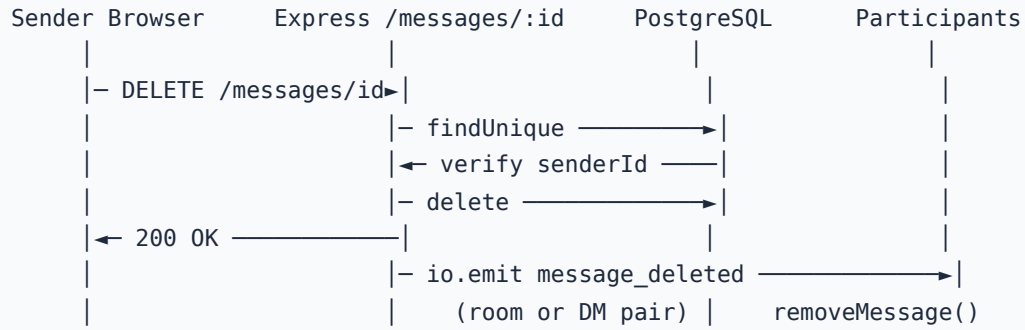
8.3 Group Message Sequence



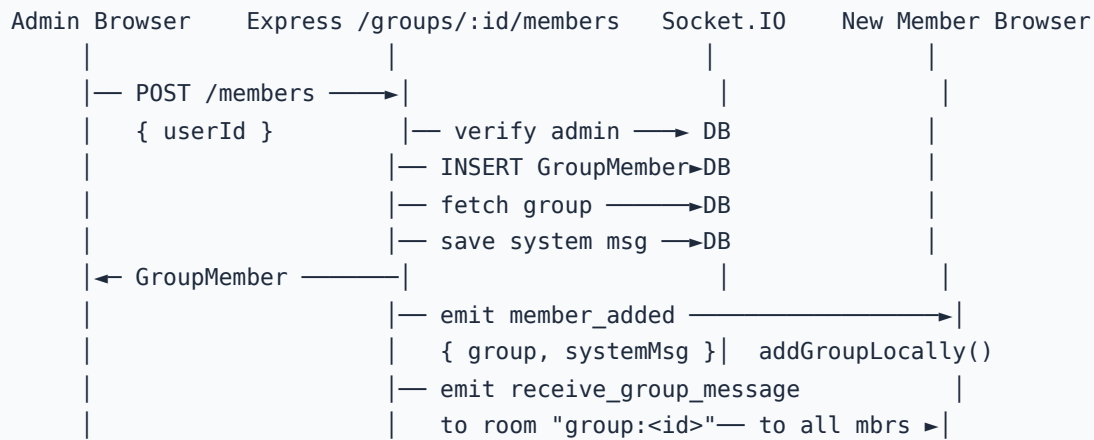
8.4 Socket Connection & Heartbeat



8.5 Delete Message Sequence



8.6 Add Group Member Sequence



9. Environment Variables

SERVER — SERVER/.ENV

Variable	Example	Required
PORT	5000	No (default 5000)
DATABASE_URL	postgresql://user:pw@localhost:5432/jigchat	Yes
REDIS_URL	redis://localhost:6379	No (default shown)
JWT_SECRET	long-random-string	Yes
ENCRYPTION_KEY	64-char hex (32 bytes)	Yes

FRONTEND — FRONTEND/.ENV

Variable	Example	Required
VITE_API_BASE_URL	http://localhost:5000/api	No (default shown)
VITE_SOCKET_URL	http://localhost:5000	No (default shown)

10. Key Features

Feature	Implementation
Real-time DM	Socket.IO <code>send_message</code> / <code>receive_message</code>
Real-time group chat	Socket.IO rooms (<code>group:<id></code>)
Message encryption	AES-256-GCM at rest, decrypted on every fetch
Online presence	Redis key with 90s TTL, refreshed by 30s heartbeat
Typing indicators	<code>typing</code> / <code>stop_typing</code> events with 2s auto-stop
Unread badges	Per-user and per-group counters in Zustand
Read receipts	<code>readAt</code> DB column + <code>mark-read</code> endpoint
Favorites	<code>user_favorites</code> table, toggled via REST
Hidden contacts	<code>hidden_contacts</code> table, auto-cleared on DM
Group management	Admin role in <code>group_members</code> , Socket broadcasts on changes
Search	ILIKE query on users + messages, scroll + highlight in thread
Message deletion	Sender-only; <code>message_deleted</code> broadcast to all parties
Profile panel	Right-side slide-in on avatar / name click
Fullscreen avatar viewer	Zoom, pan, reset via <code>AvatarViewer</code> component
Session security	Redis-backed JWT invalidation — logout is instant
Responsive layout	Tailwind breakpoints; sidebar hides on mobile